JC4003 Natural Language Processing

Lecture 12: GPT and Large Language Model

Dr Xiao Li xiao.li@abdn.ac.uk

Dr Ruizhe Li ruizhe.li@abdn.ac.uk

Dr Siwei Liu siwei.liu@abdn.ac.uk

Outline

GPT-1, GPT-2, and GTP-3:

Training process of GPT-1,2,3

Prompt Engineering

Fine-tuning GPT

Evaluating LLMs

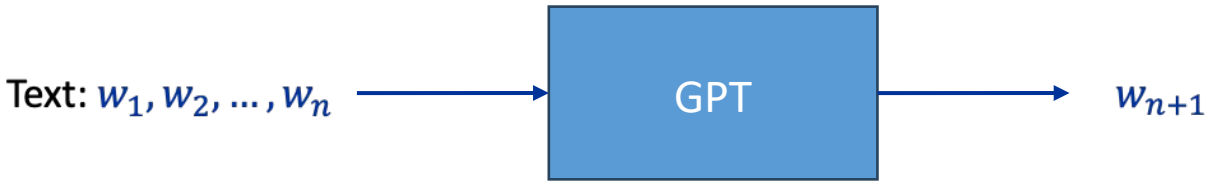
GPT

GPT

GPT (Generative Pre-trained Transformer, developed by OpenAI) is designed to generate human-like text based on a given input.

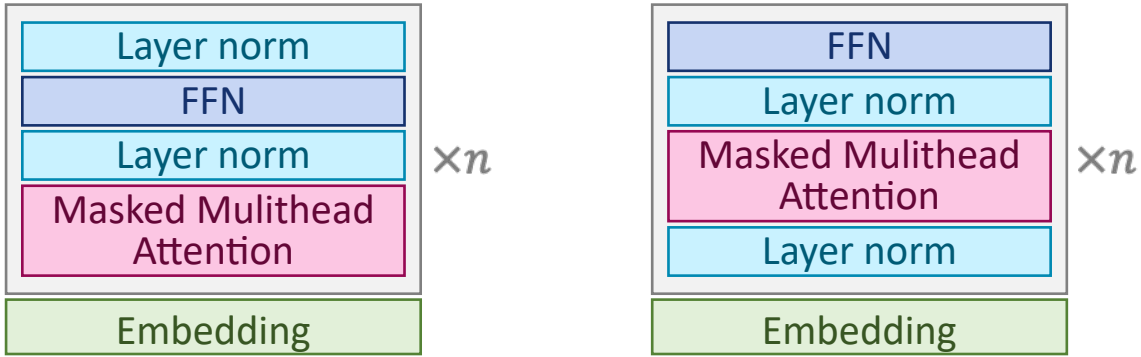
GPT models are autoregressive, meaning they predict the next word in a sequence given the previous words. Unlike bidirectional models like BERT, GPT only looks at past tokens (left-to-right context) during generation.

BERT's structure is very simple, based purely on stacked Transformer decoders (the version without the attention layer for memory).



GPT Architecture

From GPT-1 to ChatGPT, the model's architecture has remained almost unchanged (only the placement of the normalization layer was adjusted). They are all composed of purely stacked (excluding memory variants) Transformer decoders, with GPT-1 stacking 12 layers, GPT-2 stacking 48 layers, and GPT-3 stacking 96 layers.



GPT-1

GPT-2, 3, …, ChatGPT

GPT Architecture (cont.)

The specific differences between the GPT-1, 2, and 3 architectures are as follows.

	GPT1	GPT2 S	GPT2 M	GPT2 L	GPT2 XL	GPT3 S	GPT3 M	GPT3 L	GPT3 XL	GPT3 2.7B	GPT3 6.7B	GPT3 13B	GPT3 175B
Layers	12	12	24	36	48	12	24	24	24	32	32	40	96
Model dim.	768	768	1024	1280	1600	768	1024	1280	1600	2048	4096	5120	12288
Atten. head	12	12	13	20	25	12	24	32	9640	48	72	96	96
FFN Inter.	3072	3072	4096	5120	6400	3072	4096	5120	6144	8192	16348	20480	49152
Total param.	0.12B	0.12B	0.35B	0.76B	1.5B	0.13B	0.35B	0.76B	1.3B	2.7B	6.7B	13B	175B

GPT vs. BERT

Tokenisation: BERT and GPT use similar tokenisation method, splitting rare words into subword units.

Architecture: BERT uses a bidirectional Transformer encoder, looking at the context of a word from both directions. GPT uses a unidirectional Transformer decoder, predicting the next word in a sequence based on the previous words.

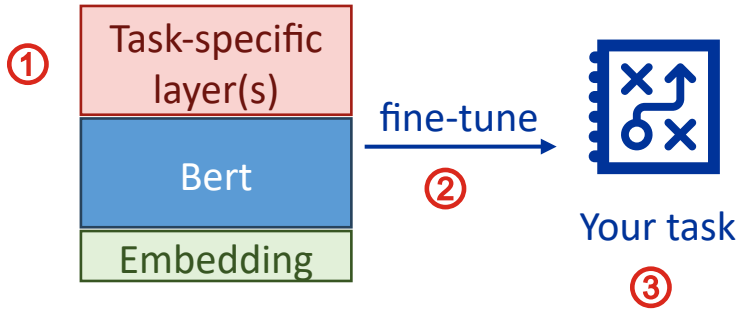
Training Objective: BERT has two objectives, MLM and NSP. GTP is only trained to predict the next word in a sentence, given all the previous words (no masking, etc.).

Applications: BERT is mainly for understanding tasks. GPT focuses on generation.

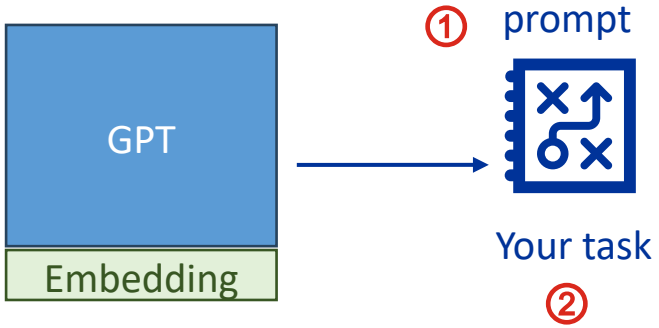
Zero-shot Learning

GPT is designed with zero-shot learning in mind, rather than based on fine-tuning like BERT. This means that it aims to perform a wide variety of tasks without fine-tuning.

To use BERT:



To use GPT (2,3,…):



Prompt

A prompt is the input provided to a language model, consisting of text or instructions, that guides the model in generating a response or performing a specific task.

Without a prompt, GPT will attempt to continue the input text. This is because GPT is trained in this way, constantly predicting what the next word should be, e.g.:

Input: Your NLP project deadline is coming

Output: up soon, and there's still quite a bit of work to be done. It's time to finalise the model, run those last few tests, and ensure everything is thoroughly documented ...

Prompt (cont.)

When using a prompt, you include various cues in the input, allowing GPT to generate output based on the combined semantics of the original input and the prompt. This way, the output can meet the requirements of the prompt, e.g.:

prompt

Input: Translated into Chinese and Japanese: Your NLP project deadline is coming

Output: **Chinese**: 你的 NLP 项目截止日期快到了。

Japanese: あなたの NLP プロジェクトの締め切りが近づいています。

Prompt with Few-shot Learning

A prompt can be very simple or very complex. You can even provide a few examples within the prompt, and GPT's reasoning ability allows it to infer the logic of the examples during the generation phase and continue generating text in line with that logic. This approach is known as few-shot learning / in-context learning, e.g.:

Input:

Examples:
The Old Man and the Sea => 
Anna Karenina => 
One Hundred Years of Solitude => 
Pride and Prejudice => 
Questions:
Journey to the West =>
Dream of Red Mansions =>
Harry Potter =>

…

Prompt with Few-shot Learning (cont.)

Input:

Examples:
The Old Man and the Sea => 
Anna Karenina => 
One Hundred Years of Solitude => 
Pride and Prejudice => 
Questions:
Journey to the West =>
Dream of Red Mansions =>
Harry Potter =>

…

Output:

Journey to the West => 
Dream of Red Mansions => 
葫芦娃 => 
甄嬛传 => 
白蛇传 => 
Lord of the Rings => 
Harry Potter => 
Tom and Jerry => 
名侦探柯南 => 

Training Process of GPT

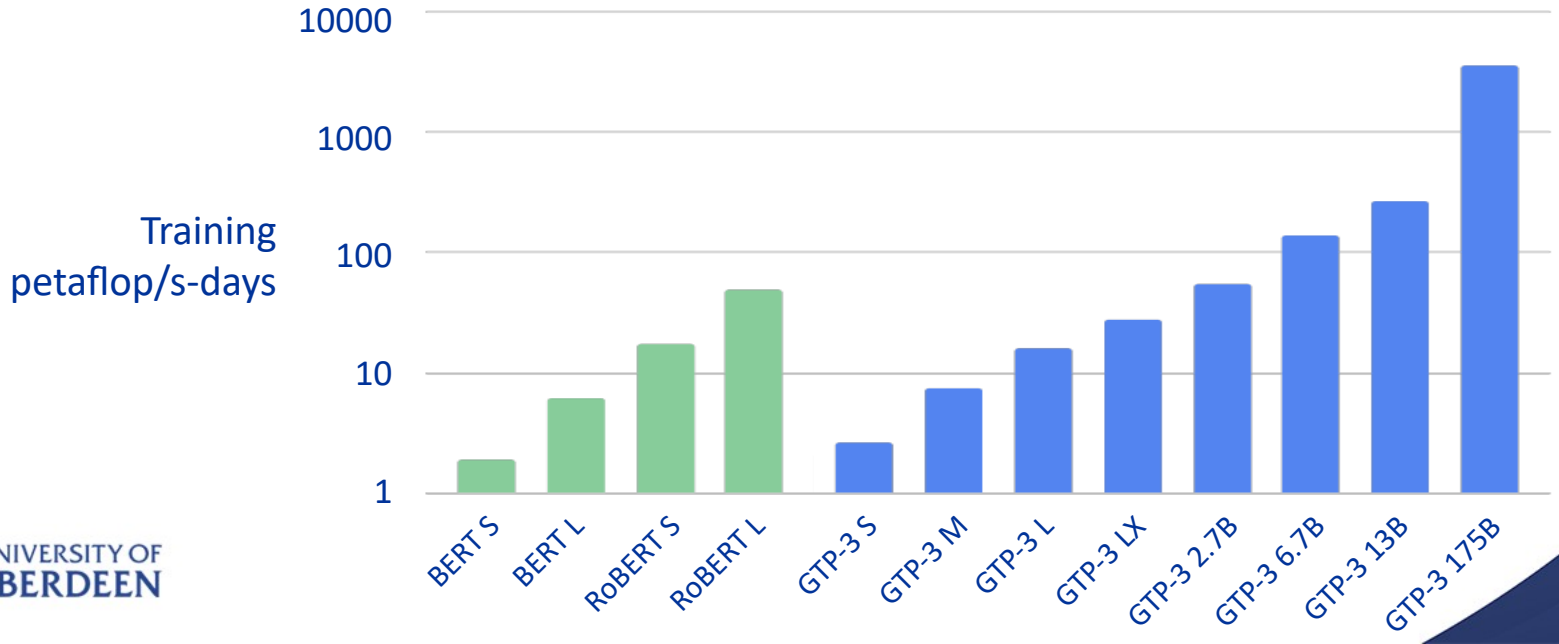
The goal is to teach the model to predict the next word in a sentence, given the previous words, i.e. the model generates the output word by word, based on the sequence it has seen so far. This is known as an autoregressive model.

The differences in the training processes of GPT-1, GPT-2, and GPT-3 primarily lie in the datasets used.

	GTP-1	GPT-2	GPT-3
Dataset	Common Crawl BooksCorpus	+ WebText	+ WebText2, Books, Wikipedia, Articles, etc.
Size	5GB	40GB	570GB
Token count	0.8 billion	8 billion	**300 billion**
Vocabulary size	40,000	50,257	50,257

Compute Requirements for Training GPTs

This is the total compute used during training various language models, measured in petaflop/s-days. As models scale up in size, the compute requirements grow exponentially.



Petaflop/s-day

A petaflop/s-day refers to the amount of computation performed at a sustained speed of 10^{15} floating-point operations per second over the course of one day, i.e.:

1 petaflop/s-day = 1,000,000,000,000,000 x 24 x 3600 = 86,400,000,000,000,000,000 (times)

That is, the total train compute of GPT-3 175B is 3.14^{23} times of floating-point operations.

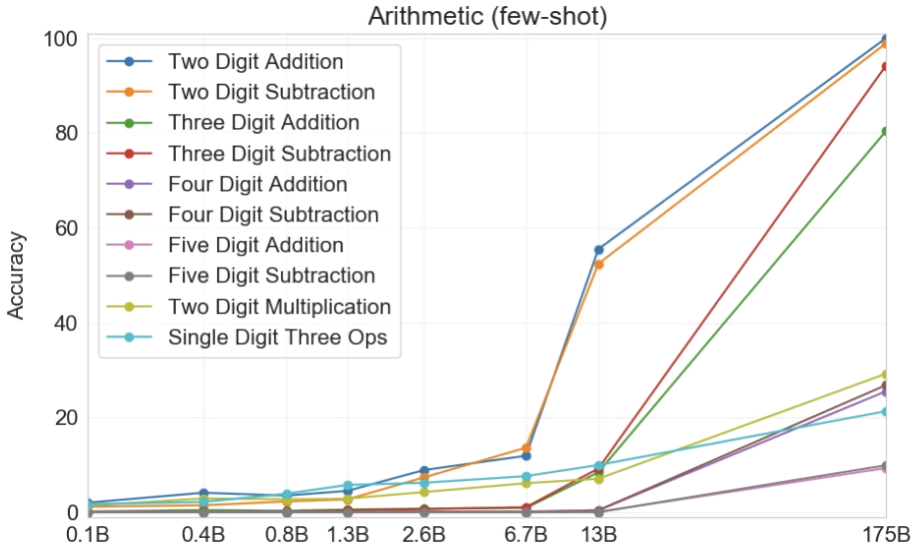
	GTP-3 S	GTP-3 M	GTP-3 L	GTP-3 XL	GTP-3 2.7B	GTP-3 6.7B	GTP-3 13B	GTP-3 175B
4090	1.6	4.5	10	17	33	84	162	2200
V100	2.1	5.9	13	22	44	111	214	2907
A100	0.8	2.4	5	9	18	45	86	1165

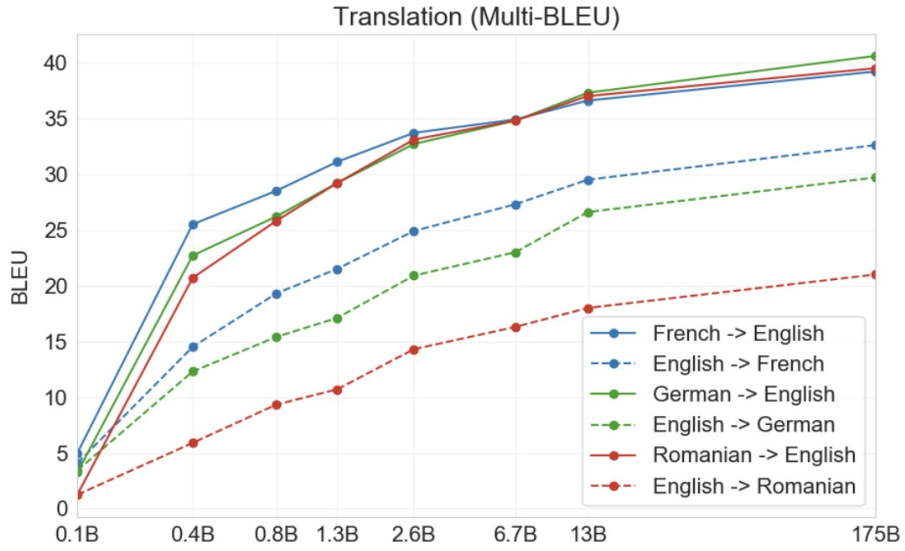
Estimated number of days required to train GPTs using different GPUs

GTP Performance Evaluation

The two charts illustrate the capabilities of GPT-3. The first chart represents the simple arithmetic abilities, while the second chart shows the translation tasks.

It can be seen that as the model's parameter size increases, its performance improves significantly.





GTP Performance Evaluation (cont.)

GPT-2 and GPT-3 share the same structure; the only differences are in the amount of training data and the number of model parameters.

Across various evaluation tasks, GPT-3 significantly outperforms GPT-2, particularly in few-shot and zero-shot learning scenarios. According to OpenAI's reports, GPT-3's performance can be several times (usually 2-4 times) better than GPT-2's, especially in tasks that require complex reasoning and comprehension.

We call this phenomenon: "brute force works wonders (大力出奇迹)".

Prompt Engineering

Prompt Engineering

When designing a prompt for GPT, you should keep the following points in mind:

Clarity

Ensure that the prompt is clear and unambiguous. E.g., rather than saying "Write an article," say "Write a 200-word article about xxx”

Few Examples (in-context learning)

Include a few examples within the prompt to help the model understand the expected style and content of the output, which can enhance the effectiveness of few-shot prompting.

Format Control

Specify the desired format to control the output. For example, "list five major applications of artificial intelligence in bullet points."

Prompt Engineering (cont.)

Context

If the task is complex, provide enough contextual information so the model can better understand it. For instance, give background information first, then ask the model to generate specific content.

Step-by-Step Prompts

Break down complex tasks into multiple steps to guide the model progressively. For example, first generate an outline, then expand on each section.

Experimentation and Iteration

Optimise the prompt through repeated experimentation and adjustments. Different prompt designs can significantly affect the results, so iterative improvement is key.

Prompt Engineering (example)

Consider the following scenario: Write a prompt that turns GPT into a self-service customer support system for after-sales services:

A Bad example:

Help me answer the user's question.

- Lack of clarity: It doesn't provide enough background or context, leaving the model unclear about what it needs to do.
- Too vague: The model doesn't understand the type of question, tone, or expected response.

Prompt Engineering (example, cont.)

A good example:

The user is asking for details about the return policy. Please explain the company's return policy in a friendly and formal tone, clearly stating that returns must be made within 30 days and the product must be unused.

- Clarity: It clearly specifies the task.
- Context: It provides the background of the user's query, helping the model better understand the task.
- Format and tone: It specifies the use of a friendly and formal tone and instructs the model to be concise, controlling the output format.

Prompt Engineering (Apple Intelligence)

Here is the prompt for Apple's email response assistant used in IOS 18:

"{{ specialToken.chat.role.system }} You are an assistant which helps the user respond to their mails. Please draft a concise and natural reply based on the provided reply snippet. Please limit the answer within 50 words. Do not hallucinate. Do not make up factual information. Preserve the input mail tone.{{

{ system control token }
{ Set a role to the LLM }
{ object }
{ specify text style }
{ for clarity }
{ ensure accuracy of information }
{ specify text style }

Fine-tuning GPT

Fine-tuning GPT

GPT is significantly more challenging to fine-tune compared to BERT due to its larger scale, generative nature, and the complexity involved in adjusting its parameters without overfitting.

Fine-tuning GPT requires more computational resources and can be prone to introducing errors. Additionally,

Fine-tuning GPT is harder to control specific outputs through (whereas it is easy for BERT).

In most cases, prompt engineering is sufficient for guiding GPT to perform specific tasks without the need for fine-tuning. Carefully designed prompts can effectively instruct GPT to generate the desired responses, making fine-tuning unnecessary for many scenarios.

Fine-tuning GPT

However, there are some tasks that do require fine-tuning to achieve the desired level of performance. Examples include:

- Domain-specific tasks: For instance, generating legal documents or medical reports, where the model needs specialised knowledge that goes beyond general prompts.
- Highly structured output generation: Such as generating complex code or scripts, where precise control over the output is necessary.
- Personalisation: Tailoring the model's responses to a specific individual's tone, style, or preferences, which may require extensive adjustments that cannot be achieved with prompts alone.

Fine-tuning GPT

There are three common methods for fine-tuning GPT:

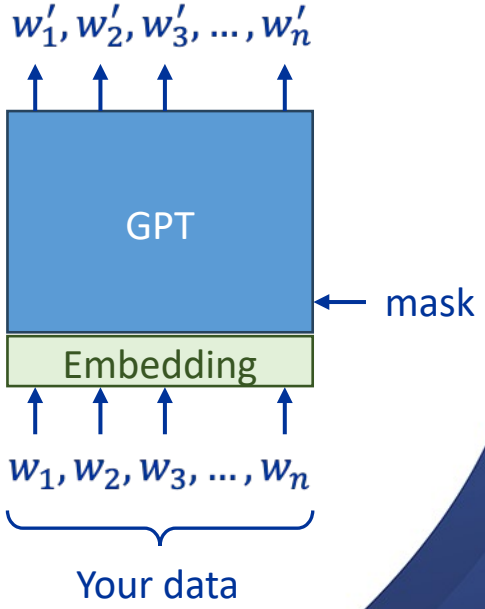
1. Basic Fine-Tuning: Like to fine-tune BERT, continuously training GPT on task-specific datasets to improve its performance in a particular domain or task.
2. P-Tuning: Fine-tuning with minimal parameters by using continuous prompts to guide GPT's performance in specific tasks.
3. Reinforcement Learning from Human Feedback (RLHF): Training GPT with human feedback to make it more aligned with user expectations, enhancing the relevance and safety of the model's responses.

Basic Fine-Tuning

Like to fine-tune BERT, you continuously train GPT on your task-specific datasets.

However, if your dataset is very small, you might encounter challenges such as overfitting. To mitigate this, you can apply regularisation methods like dropout and use a small learning rate.

The recommended sizes of the dataset for fine-turn (for GPT-3) are typically between the size of 10^5 – 10^6 samples.



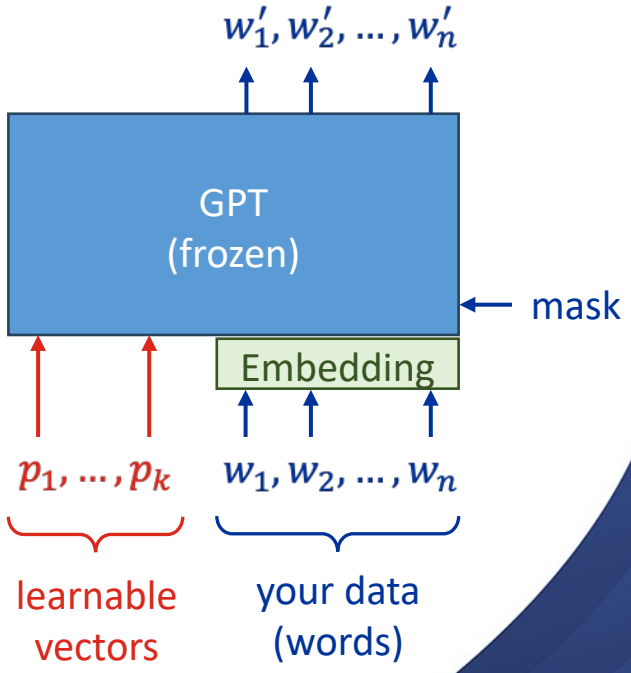
P-Tuning

P-Tuning is a method that fine-tunes pre-trained language models like GPT by introducing learnable continuous prompt vectors ($p_1, \ldots, p_k$), to effectively guide the model's behaviour, acting like encoded prompts.

Prompt vectors occupy the positions of word embeddings. During P-Tuning, both GPT and the embeddings are frozen, and the gradients passing through GPT to the prompt vectors.

By only adjusting these prompt vectors, the model adapts to the task without altering its core parameters.

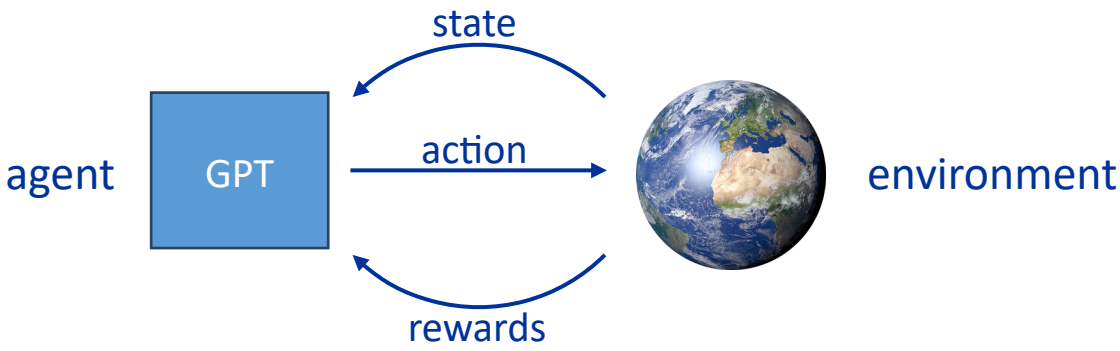
The recommended sizes of the dataset for P-Turn (for GPT-3) are typically between the size of 10^3 – 10^5 samples.



RLHF

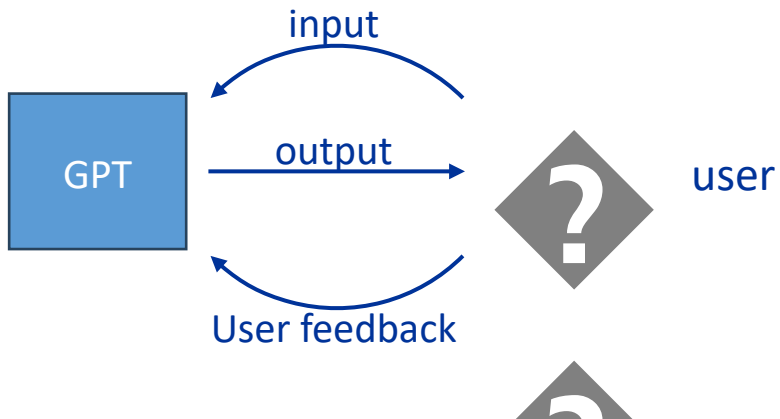
Reinforcement Learning from Human Feedback (RLHF) is a reinforcement learning process where large language models like GPT are trained to align with human preferences by incorporating feedback during the training process.

Reinforcement learning studies the interaction between an agent (LLM) and its environment, where the agent learns to make decisions by taking actions that maximise cumulative rewards based on feedback from the environment.



RLHF (cont.)

When fine-tuning GPT, the GPT model acts as the agent, the user serves as the environment, the input is the state, the output is the action, and user feedback represents the reward.

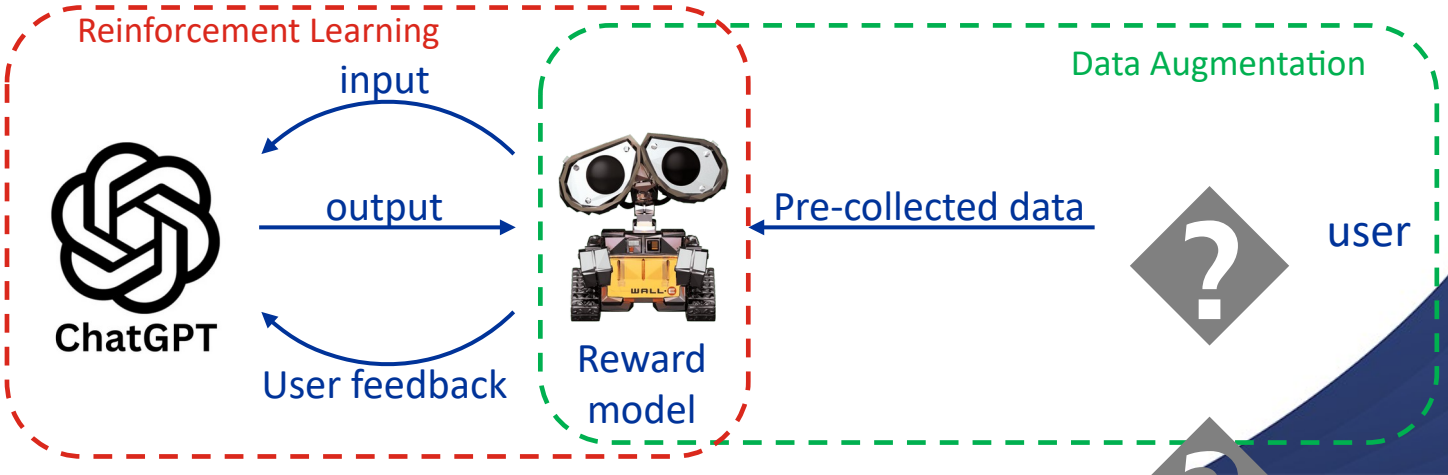


RLHF (cont.)

Although users are involved in the model fine-tuning process, they cannot continuously respond to GPT‘s actions in real-time.

To improve the efficiency of fine-tuning, RLHF collects user feedback in batches beforehand and then trains a separate reward model based on this feedback. This reward model is used to replace direct human feedback by assigning rewards to GPT's actions during training.

The way to train ChatGPT



Evaluate Language Models (part 4)

1495 UNIVERSITY OF ABERDEEN

Evaluating LLMs

Evaluating LLMs involves several key methods to assess their performance across different tasks:

- Benchmarking: Standard datasets and benchmarks like SuperGLUE, SQuAD, and TriviaQA are used to evaluate LLMs on language understanding, question answering, and reasoning tasks.
- Human Evaluation: Human evaluators assess the model's outputs for quality, coherence, and relevance.
- Human Exams: LLMs are tested on real-world exams (e.g., SAT, GRE, or China's Gaokao), which challenge them with complex reasoning, comprehension, and problem-solving tasks similar to what humans face.

Benchmarking

There are several standard datasets and benchmarks are commonly used. Each dataset focuses on different capabilities:

- SuperGLUE: Tests general language understanding, reasoning, and inference with tasks like Boolean Question Answering and Natural Language Inference.
- SQuAD (Stanford Question Answering Dataset): Focuses on reading comprehension by asking models to extract answers from paragraphs.
- TriviaQA: Evaluates open-domain question answering, requiring models to retrieve and generate answers from longer documents.
- PIQA: Assesses commonsense reasoning, specifically in physical interactions and everyday scenarios.

Benchmarking (cont.)

- MMLU (Massive Multitask Language Understanding): This benchmark evaluates LLMs across a wide range of academic and professional subjects, testing their ability to understand and generate text in diverse domains like history, mathematics, and law.
- C-Eval: A benchmark focused on evaluating LLMs on tasks relevant to the Chinese language and context, covering areas like literature, geography, and common knowledge.
- AGI Eval: Designed to assess models' performance across a variety of tasks that test general intelligence, problem-solving, and reasoning.
- GSM8K: A benchmark focused on grade-school math problems, testing a model's arithmetic, algebra, and word problem-solving capabilities.

Human Evaluation

Human Evaluation is to let human evaluators manually review and score the model's outputs.

By asking human evaluators, you can focus on any aspect of the model's output, such as coherence, relevance, fluency, and accuracy of the responses. Unlike automated metrics, human evaluation can capture nuances like creativity, context appropriateness, and subtle errors that machines might miss.

Human evaluation is typically conducted through various detailed questionnaires.

Human Exams

There has been significant work on using large language models (LLMs) to take human exams, testing their capabilities in real-world scenarios. Researchers have evaluated LLMs on exams such as the SAT, GRE, GMAT, even medical licensing exams, etc.

These tests challenge models with complex reasoning, language comprehension, and problem-solving tasks, similar to those faced by human test-takers.

The goal is to assess the models' general intelligence, adaptability, and practical knowledge across various domains.

Human Exams (cont.)

Here's an example of a large model taking the Chinese Gaokao (college entrance exam).

The evaluation was conducted using the National New Curriculum Standard I test paper (2024). All participating open-source models were released before the Gaokao exam to ensure the assessment was "closed-book." Additionally, the scores were manually evaluated by teachers with experience in grading Gaokao exams.

语数外得分情况				
模型	语文 (满分150)	数学 (满分150)	英语 (满分120)	总分 (满分420)
Qwen2-72B	124	70	109	303
GPT-4o	111.5	73	111.5	296
InternLM2-20B-WQX	112	75	108.5	295.5
Qwen2-57B	99.5	58	96.5	254
Yi-1.5-34B	97	29	104.5	230.5
GLM-4-9B	86	49	67	202
Mixtral 8x22B	77.5	21	86.5	185

Human Exams (cont.)

The detailed marks:

英语各题型得分情况						
模型	阅读理解（满分37.5分）	七选五（满分12.5分）	完型填空（满分15分）	语法填空（满分15分）	作文（满分40分）	总分（满分120）
GPT-4o	37.5	10	14	15	35	111.5
Qwen2-72B	35	12.5	14	13.5	34	109
InternLM2-20B-WQX	37.5	10	15	13.5	32.5	108.5
Yi-1.5-34B	35	10	11	13.5	35	104.5
Qwen2-57B	35	10	9	15	27.5	96.5
Mixtral 8x22B	37.5	5	2	9	33	86.5
GLM-4-9B	35	0	6	6	20	67

Human Exams (cont.)

The detailed marks:

数学各题型得分情况					
模型	单选择（满分40分）	多选题（满分18分）	填空题（满分15分）	问答题（满分77分）	总分（满分150）
InternLM2-20B-WQX	30	9	10	26	75
GPT-4o	35	6	10	22	73
Qwen2-72B	30	12	10	18	70
Qwen2-57B	35	9	5	9	58
GLM-4-9B	30	7	0	12	49
Yi-1.5-34B	20	5	0	4	29
Mixtral 8x22B	10	0	0	11	21

Human Exams (cont.)

The detailed marks:

语文各题型得分情况							
模型	现代文阅读（满分35分）	文言文阅读（满分22分）	古诗文阅读（满分9分）	名句默写（满分6分）	语言文字运用（满分18分）	作文（满分60分）	总分（满分150）
Qwen2-72B	31	19	9	6	9	50	124
InternLM2-20B-WQX	30	17	6	6	7	46	112
GPT-4o	32	10	8	2	9	50.5	111.5
Qwen2-57B	27	12	7	6	2	45.5	99.5
Yi-1.5-34B	28	8	5	2	4	50	97
GLM-4-9B	21	6	8	6	4	41	86
Mixtral 8x22B	18	3	7	2	3	44.5	77.5

Conclusion

GPT Overview: GPT models are autoregressive, generating text by predicting the next word based on previous tokens.

Architectural Progression: GPT evolved from GPT-1 to GPT-3, with increasing layers and parameters, enhancing its performance.

Prompt Engineering: Effective use of prompts can guide GPT to generate desired outputs, minimising the need for fine-tuning.

Fine-tuning: While GPT is often used with prompts, fine-tuning is necessary for domain-specific or highly structured tasks.

Evaluation: LLMs are evaluated using benchmarks like SuperGLUE and human exams, testing their reasoning, comprehension, and problem-solving abilities.